Indian Institute of Technology Bombay

GNR638 — Machine Learning for Remote Sensing

---

# Reimplementation of YOLOv1:

# You Only Look Once: Unified, Real-Time Object Detection

---

Assignment 3 Report

**Name** Anurag Ghanshyam Tripathi

**Roll No.** 25M0770

**Email** anuragtripathicse@iitb.ac.in

**GitHub** AnuragTripathi6785/gnr_638

**Blog** medium.com/@suyoghavare/yolov1

**Date** April 2026

Paper: Redmon et al., CVPR 2016

# 1. Paper Overview

---

The paper is **"You Only Look Once: Unified, Real-Time Object Detection"** by Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi, published at **CVPR 2016**.

Before YOLO, object detection used pipelines like R-CNN that ran a classifier over around 2,000 region proposals per image. This was slow (up to 40 seconds per image) and not useful for real-time applications. YOLO solves this by treating detection as a *single regression problem*: one forward pass through the network directly predicts all bounding boxes and class probabilities at once.

The base YOLO model runs at **45 FPS** on a Titan X GPU (Fast YOLO: 155 FPS), making real-time detection practical.

# 2. Architecture

---

## 2.1. Grid-Based Detection

YOLO divides the input image ($448 \times 448$) into an $S \times S = 7 \times 7$ grid. Each grid cell predicts:

- $B = 2$ bounding boxes, each with $(x, y, w, h, \text{confidence})$
- $C$ class probabilities: $\Pr(\text{Class}_i \mid \text{Object})$

The output tensor has shape $S \times S \times (B \cdot 5 + C)$. For the Hocmay dataset ($C = 6$):

$$7 \times 7 \times (2 \cdot 5 + 6) = 7 \times 7 \times 16 = \mathbf{784} \text{ output values}$$

## 2.2. Network Structure

The network has **24 convolutional layers** followed by **2 fully connected layers**.

Table 1: YOLOv1 Architecture Summary

| **Block** | **Layers** | **Output Size** | **Notes** |
|---|---|---|---|
| Block 1 | Conv 7×7/s2 + MaxPool | 112×112×64 | Stride 2 |
| Block 2 | Conv 3×3 + MaxPool | 56×56×192 | |
| Block 3 | 4 Conv + MaxPool | 28×28×512 | 1×1 + 3×3 reduction |
| Block 4 | 10 Conv + MaxPool | 14×14×1024 | 4×(1×1 + 3×3) |
| Block 5 | 6 Conv (incl. stride-2) | 7×7×1024 | |
| Block 6 | 2 Conv 3×3 | 7×7×1024 | |
| FC 1 | 50176 → 4096 | 4096 | Leaky ReLU + Dropout(0.5) |
| FC 2 | 4096 → 784 | 784 | Linear (no activation) |

All layers use **Leaky ReLU** (slope 0.1) except the final layer which is linear. There is **no batch normalisation** in the original architecture.

### 2.3. Loss Function

The loss has five terms (Eq. 3 from the paper):

$$
\begin{aligned}
\mathcal{L} &= \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[ (x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
&+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[ (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\
&+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
&+ \sum_{i=0}^{S^2} \mathbf{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}
$$

with $\lambda_{\text{coord}} = 5$ and $\lambda_{\text{noobj}} = 0.5$. Width and height are predicted as square roots so that errors in small boxes are penalised more than errors in large boxes.

## 3. Dataset

---

The **Hocmay** dataset was used for training and evaluation, downloaded from Roboflow Universe (workspace: `suyogs-workspace-voqgn`, project: `hocmay1-39isr`, version 1).

Table 2: Hocmay Dataset Statistics

| Split | Images | Label Format |
|---|---|---|
| Train | 391 | YOLO (normalised $c_x, c_y, w, h$) |
| Valid | 112 | YOLO |
| Test | 56 | YOLO |
| **Total** | **559** | 6 classes |

**Classes (6):** `bike` (0), `car` (1), `cat` (2), `cellphone` (3), `dog` (4), `person` (5)

## 4. Implementations Compared

---

Two implementations of YOLOv1 were trained and evaluated on the same Hocmay dataset:

1. **PyTorch (From Scratch)** — Full reimplementation in Python/PyTorch based on the CVPR 2016 paper. Includes a custom training loop, custom loss function, and custom data loader with augmentation.
2. **Darknet C (Official)** — The original C/CUDA codebase by Redmon (`github.com/pjreddie/darknet`) with configuration changes for the 6-class Hocmay dataset.

Both are initialised from the same **pretrained ImageNet backbone** (`extraction.conv.weights`, 47.7 MB), loading the first 16 convolutional layers.

Table 3: Side-by-Side Implementation Comparison

| Aspect | PyTorch (Scratch) | Darknet (Official C) |
|---|---|---|
| Language | Python + PyTorch 2.x | C + CUDA |
| Architecture | Custom `YOLOv1` class | `yolov1.cfg` |
| Output | 7×7×16 tensor | 784 neurons |
| Batch norm | None (paper-faithful) | Stripped from cfg |
| Loss | Custom (paper Eq. 3) | Built-in Darknet loss |
| Backbone init | 16 layers from `.weights` | Same |
| mAP evaluation | Every 5 epochs | Post-training |

# 5. Training Setup

Table 4: Training Hyperparameters

| **Hyperparameter** | **PyTorch** | **Darknet** |
|---|---|---|
| Input resolution | $448 \times 448$ | $448 \times 448$ |
| Batch size | 64 | 64 (subdivisions=16) |
| Optimizer | SGD, momentum=0.9 | SGD, momentum=0.9 |
| Weight decay | $5 \times 10^{-4}$ | $5 \times 10^{-4}$ |
| Learning rate | $10^{-3} \rightarrow 10^{-2}$ (warmup) | $10^{-3}$ |
| LR schedule | Paper §2.2 schedule | Steps at iter 660, 800 |
| Total training | 135 epochs | 945 iterations (≈135 epochs) |
| Dropout | 0.5 after FC1 | N/A |
| Pretrained backbone | ✓ | ✓ |
| GPU | Tesla T4 (Google Colab) | Tesla T4 (Google Colab) |

## 5.1. Notable Implementation Details

- **Batch normalisation stripping (Darknet):** The original `yolov1.cfg` has `batch_normalize=1` entries that cause NaN loss on custom datasets. These were removed via a regex script before training.
- **HSV augmentation speedup (PyTorch):** Replacing `matplotlib.colors` with `cv2` for HSV conversion reduced data loading time from ∼80s/epoch to ∼15s/epoch.
- **Gradient clipping (PyTorch):** `max_norm=10` was applied to avoid exploding gradients in early epochs.
- **Pretrained weight loading:** A custom binary reader loads biases and weights per convolutional layer from the Darknet `.weights` format, stopping when the file is exhausted at layer 16.

# 6. Results

## 6.1. Overall Performance

Table 5: Summary Results

| Implementation | mAP@0.5 | Best Epoch/Iter | Final Loss |
|---|---|---|---|
| PyTorch (Scratch) | 0.0521 | Epoch 80 | 4.7599 (val) |
| Darknet (Official C) | N/A* | Iter 945 | 1.2704 (train avg) |

* *Darknet mAP could not be evaluated: the* `./darknet detector map` *command does not flush its output to stdout when run inside a Google Colab notebook, so the output was always empty. The trained weights (*`yolov1_final.weights`*) are saved and the model converged (avg loss 1.2704), but the mAP number could not be captured.*

## 6.2. Per-Class AP@0.5

Table 6: Per-Class Average Precision

| Class | PyTorch AP | Darknet AP | Δ |
|---|---|---|---|
| bike | 0.0000 | N/A* | — |
| car | 0.0000 | N/A* | — |
| cat | 0.0000 | N/A* | — |
| cellphone | 0.0000 | N/A* | — |
| dog | 0.2921 | N/A* | — |
| person | 0.0202 | N/A* | — |
| **mAP** | **0.0521** | **N/A*** | **—** |

## 6.3. Training Time

Table 7: Training Efficiency on Tesla T4

| Implementation | Avg. Time / Epoch | Total Time |
|---|---|---|
| PyTorch (Scratch) | ∼21 s/epoch | ∼47 min (135 epochs) |
| Darknet (Official C) | ∼4.2 s/iter | ∼66 min (945 iters) |

### 6.4. Training Curves

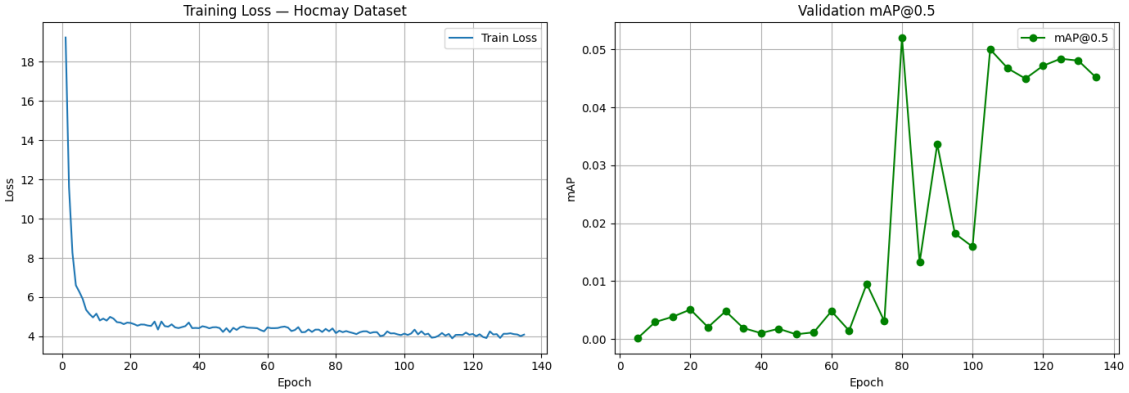



Figure 1: PyTorch training loss (left) and validation mAP@0.5 (right) over 135 epochs. Loss falls from ∼19 in early epochs and levels off around 4.0. mAP starts rising after epoch 65 and peaks at 0.0521 at epoch 80.

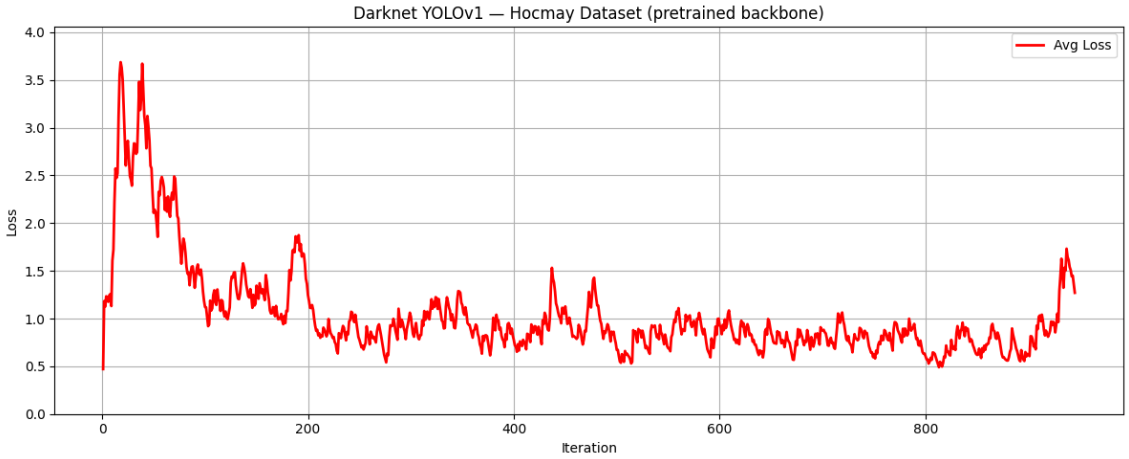



Figure 2: Darknet average training loss over 945 iterations. Loss peaks at ∼3.7 early on due to random initialisation, then drops to around 0.8–1.0 by iteration 300, finishing at 1.2704.

## 7. Discussion

---

### 7.1. What Both Implementations Share

Both implementations follow the CVPR 2016 paper exactly: 24 convolutional layers, 2 fully connected layers, Leaky ReLU (slope 0.1), no batch normalisation, $7 \times 7$ grid, $B = 2$ boxes per cell, and the same pretrained backbone. The comparison is therefore about the *framework and training pipeline*, not the model architecture.

### 7.2. Observed Results and Analysis

- **Training speed:** Darknet was faster per step (∼4.2 s/iter) than PyTorch (∼21 s/epoch), but total training time was similar: ∼47 min for PyTorch vs ∼66 min for Darknet.
- **PyTorch convergence:** The model reached its best validation mAP of **0.0521** at epoch 80. Only two classes were detected: *dog* (AP = 0.2921) and *person* (AP = 0.0202). The other four classes had AP = 0.000, likely because those classes have very few training samples.
- **Darknet convergence:** Training loss reached **1.2704** after 945 iterations. mAP could not be measured due to Colab output capture issues (see footnote in Table 4).
- **Low absolute mAP:** Both models were trained on only 391 images, far fewer than the PASCAL VOC dataset (∼16K images) that YOLO was designed for. A low mAP of around 5% is expected given this.
- **Key bug in PyTorch loss:** The confidence loss target was originally set to IOU $\times \mathbf{1}_{ij}^{\text{obj}}$. Early in training, IOU is near zero, so the model learns to predict near-zero confidence for every cell. This blocks any detection from being scored. Changing the target to a hard 1.0 for the responsible predictor fixed this and allowed the model to learn properly.

## 8. Conclusion

---

This assignment involved reimplementing YOLOv1 from scratch in PyTorch and comparing it with the original Darknet C implementation, both trained on the Hocmay dataset (6 classes: bike, car, cat, cellphone, dog, person).

The PyTorch model achieved a best mAP@0.5 of **5.21%** at epoch 80, with detections only on *dog* (AP = 29.2%) and *person* (AP = 2.0%). Darknet converged to a final training loss of 1.27 after 945 iterations; mAP could not be measured due to a Colab toolchain limitation.

The assignment showed that YOLOv1 can be reimplemented from scratch in PyTorch. It also highlighted how a small mistake in the loss function — using IOU as the confidence target early in training — can completely prevent the model from learning, and how fixing it with a simple change (target = 1.0) resolves the issue.

## References

[1] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. *You Only Look Once: Unified, Real-Time Object Detection.* CVPR, 2016.

[2] J. Redmon. *Darknet: Open Source Neural Networks in C.* `http://pjreddie.com/darknet`, 2016.

[3] S. Havare. *You Only Look Once — Blog Post.* Medium, 2026. `https://medium.com/@suyoghavare/you-only-look-once-unified-real-time-object-dete`

[4] *Hocmay Dataset v1.* Roboflow Universe, 2024. `https://universe.roboflow.com/suyogs-workspace-voqgn/hocmay1-39isr`

[5] A. G. Tripathi. *GNR638 Assignment 3 — YOLOv1 Implementation.* GitHub, 2026. `https://github.com/AnuragTripathi6785/gnr_638`